SkillBridge

Student Learning and Course Management System

Django Project Proposal, Requirements and Implementation Documentation

Project type	Web application
Backend	Python + Django
Database	SQLite for development; PostgreSQL recommended for production
Suggested UI	Django Templates + Bootstrap 5
Optional API layer	Django REST Framework
Document version	1.0
Prepared	29 October 2026

This document describes a proposed Django project. It is a project specification and implementation guide, not a claim that a runnable source-code repository has already been generated.

1. Project Overview

SkillBridge is a web-based learning and course management platform. It helps learners discover courses, enroll, access learning materials, track progress, and ask questions. Instructors manage courses and learning content, while administrators manage users and platform data.

2. Problem Statement

Course information, learning resources, enrollment records, and learner progress are often spread across separate tools or manual records. SkillBridge brings these workflows into one application with role-based access and a structured database.

3. Objectives

- Provide a central catalogue of available courses.
- Allow registered learners to enroll in courses and view their enrolled-course list.
- Allow instructors to create and maintain courses and learning materials.
- Track lesson completion and display learner progress.
- Provide a question-and-answer area for course-related doubts.
- Give administrators tools to manage users, courses, and enrollments.

4. User Roles

Administrator

- Manage users and permissions through Django Admin.
- Review, publish, unpublish, or remove inappropriate course content.
- Monitor course and enrollment records.

Instructor

- Create and edit courses owned by the instructor.
- Add lessons and learning resources.
- View enrolled learners and respond to course questions.

Learner

- Register, log in, and update profile details.
- Browse course catalogue and enroll.
- Open lessons, mark lessons complete, and view progress.
- Post questions and read instructor replies.

5. Functional Requirements

- Authentication: registration, login, logout, password reset, and permission checks.
- Course catalogue: list, search, filter by category, and view course details.
- Enrollment: enroll a learner once per course; prevent duplicate enrollments.
- Course authoring: instructors create, update, publish, and archive their own courses.
- Lessons: add ordered lessons with text content and optional resource links or uploaded files.

- Progress: store lesson completion and calculate a percentage for each enrollment.
- Q&A;: learners ask questions; instructors can answer or mark a question resolved.
- Administration: manage categories, users, courses, and reports.

6. Non-Functional Requirements

- Security: use Django authentication, CSRF protection, server-side validation, and permission checks.
- Usability: responsive pages, clear validation messages, and accessible form labels.
- Maintainability: separate functionality into Django apps and use migrations.
- Reliability: validate uploaded files and handle missing or unauthorized records safely.
- Performance: add database indexes to common filters as data grows.

7. Recommended Technology Stack

Core: Python 3.12+ (or a version supported by the chosen Django release), Django, SQLite for local development. **Production:** PostgreSQL, environment variables for secrets, HTTPS, and a production WSGI/ASGI server. **Frontend:** Django Templates and Bootstrap 5. **Optional:** Django REST Framework for APIs, Pillow for image uploads, and a cloud object store for uploaded files.

8. Suggested Django App Structure

```
skillbridge/
■■■ manage.py
■■■ config/              # settings, urls, asgi, wsgi
■■■ accounts/            # profile and account-related views
■■■ courses/             # categories, courses, lessons
■■■ enrollments/         # enrollments and progress
■■■ discussions/         # learner questions and instructor replies
■■■ templates/
■■■ static/
■■■ media/               # local development uploads only
■■■ requirements.txt
```

The exact app split can be simplified for a small academic project. Keep business logic in models/services and access control in views or permissions.

9. Database Design

Model	Important fields	Relationship / purpose
User	Django built-in User; optional profile fields	Authentication and role/group membership
Category	name, slug, description	One category has many courses
Course	title, slug, description, thumbnail, instructor, category, status, created_at	Instructor and category are foreign keys
Lesson	course, title, content, order, resource_file, resource_url	Many lessons belong to one course
Enrollment	learner, course, enrolled_at, status	Unique learner + course pair
LessonProgress	enrollment, lesson, completed, completed_at	Tracks completion per enrolled learner and lesson
Question	course, learner, title, body, created_at, is_resolved	Course-related learner questions
Answer	question, instructor, body, created_at	One question may have multiple answers

Recommended constraints: unique Category.slug, Course.slug, and (Enrollment.learner, Enrollment.course); Lesson.order should be unique within a course if strict ordering is required. Use database constraints as well as form validation.

10. Main Pages / Screens

- Home page: platform introduction and featured published courses.
- Course catalogue: search, category filters, pagination.
- Course detail: description, instructor, lesson outline, and enroll action.
- Learner dashboard: enrolled courses, completion percentages, recent questions.
- Instructor dashboard: owned courses, course editor, lesson editor, learner list, Q&A.;
- Lesson page: lesson content and completion action.
- Profile page: account and profile information.
- Admin site: internal management of models and users.

11. URL Plan

/	Home page
/accounts/register/	Learner registration
/accounts/login/ and /accounts/logout/	Authentication
/courses/	Published course catalogue
/courses/<slug>/	Course detail
/dashboard/	Role-aware dashboard
/courses/<slug>/enroll/	Enroll in a course
/lessons/<id>/	Lesson detail
/lessons/<id>/complete/	Mark lesson complete
/discussions/	Course questions and answers
/admin/	Django administration

These routes are proposed examples. Adapt names and route converters to the actual implementation.

12. Business Rules and Permissions

- Only published courses appear in the public catalogue.
- Only the course owner or an authorized administrator can edit a course.
- A learner cannot enroll in the same course more than once.
- Only enrolled learners can access protected course lessons.
- Learners can mark progress only for lessons in courses they are enrolled in.
- Uploaded files must be validated for size and type; never trust a filename or client-provided content type.
- Do not expose secret keys or debug mode in production.

13. Development Setup

Create and activate a virtual environment, install Django, start the project and apps, configure settings, then migrate the database.

```
# Windows PowerShell
py -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install django pillow

django-admin startproject config .
python manage.py startapp accounts
python manage.py startapp courses
python manage.py startapp enrollments
python manage.py startapp discussions

python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

Add each app to INSTALLED_APPS in config/settings.py. Configure templates, static files, media settings, database settings, and project URLs. Define models before running makemigrations. The commands above establish a starting environment; they do not create all business features automatically.

14. Suggested Implementation Sequence

- Set up the project, settings, app folders, templates, and base navigation.
- Create Category, Course, and Lesson models; register them in Django Admin.
- Implement authentication and role/group setup.
- Build catalogue and course-detail views with published-course filtering.
- Implement instructor-only course and lesson forms.
- Implement enrollment with a database uniqueness constraint.
- Add lesson access checks and progress tracking.
- Build learner and instructor dashboards.
- Add Q&A;, search, pagination, validation, and responsive styling.
- Write tests, configure deployment settings, and document backup/restore.

15. Testing Checklist

- A new user can register and log in; invalid credentials are rejected.
- Published courses are visible; draft courses are hidden from learners.
- An instructor cannot edit another instructor's course.
- Duplicate enrollment is prevented, including concurrent submissions.
- A non-enrolled learner cannot access a protected lesson.
- Progress is calculated correctly when lessons are completed.
- Invalid uploads and invalid form data are rejected.
- CSRF protection and authentication requirements are active.
- Admin users can manage records; ordinary users cannot access admin-only actions.
- Run automated tests with python manage.py test.

16. Security and Deployment Notes

- Set DEBUG=False in production and define ALLOWED_HOSTS.
- Keep SECRET_KEY, database credentials, and email credentials in environment variables.
- Use HTTPS and secure cookie settings behind a correctly configured proxy.
- Run python manage.py check --deploy before release.
- Run migrations during deployment and collect static files with collectstatic.
- Use PostgreSQL for production and schedule database/media backups.
- Serve user uploads as untrusted content; restrict allowed file types and size.
- Use least-privilege database credentials and keep dependencies updated.

17. Future Enhancements

- REST API using Django REST Framework and token/session authentication.
- Course ratings, certificates, announcements, and email notifications.
- Payment integration for paid courses.
- Quizzes, assignment submissions, grading, and analytics.
- Video hosting integration and asynchronous tasks for notifications.
- Deployment with Docker and CI/CD.

18. CKEditor-Ready Project Title and Description

Suggested title field value:

```
SkillBridge – Student Learning and Course Management System
```

Suggested description field value (HTML). This can be stored as the editor's HTML data, provided your CKEditor configuration permits these elements:

```
<h2>About SkillBridge</h2>
<p><strong>SkillBridge</strong> is a web-based learning and course management
platform built with Python and Django. It connects learners and instructors
in one place and makes course delivery easier to manage.</p>
<h3>Key Features</h3>
<ul>
  <li>Browse and search published courses.</li>
  <li>Enroll in courses and access lessons.</li>
  <li>Track lesson completion and learning progress.</li>
  <li>Allow instructors to manage courses and learning materials.</li>
  <li>Ask course-related questions and receive instructor responses.</li>
  <li>Provide administrators with user and content management tools.</li>
</ul>
<p>The platform uses role-based permissions to protect course content and
keep learner and instructor workflows organized.</p>
```

Important: store the HTML only if the application is designed to accept rich text. Sanitize or safely render editor content, restrict who can edit it, and do not mark user-supplied HTML as safe without appropriate sanitization.

19. Deliverables

- Django project source code with separate apps.
- Database models and migrations.
- Templates and responsive user interface.

- Authentication and role-based permissions.
- Course, lesson, enrollment, progress, and Q&A; features.
- Automated tests and setup instructions.
- Deployment configuration and final user documentation.

20. Conclusion

SkillBridge is a practical Django portfolio or academic project that demonstrates authentication, CRUD operations, foreign-key relationships, permissions, file handling, database constraints, dashboards, and testing. Build and test each module incrementally, and treat this document as the baseline specification to update as the implementation evolves.